

Agentic Workflows

Date: 2025-07-21 14:55

#LECTURE

#AI

#PYTHON

LLMs as a Work Horse

Content

- AGENTS
 - MCP
 - RAG
 - WEBSITE
-

Agents

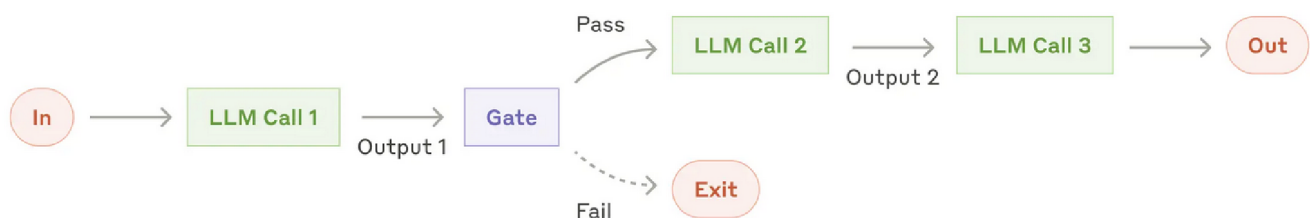
An agent is a fundamental definition in Artificial Intelligence that describes a system that can take in input from its environment and perform actions on it.

Agents then perform *workflows* which is a series of thoughts and actions to produce a desired result to the environment.

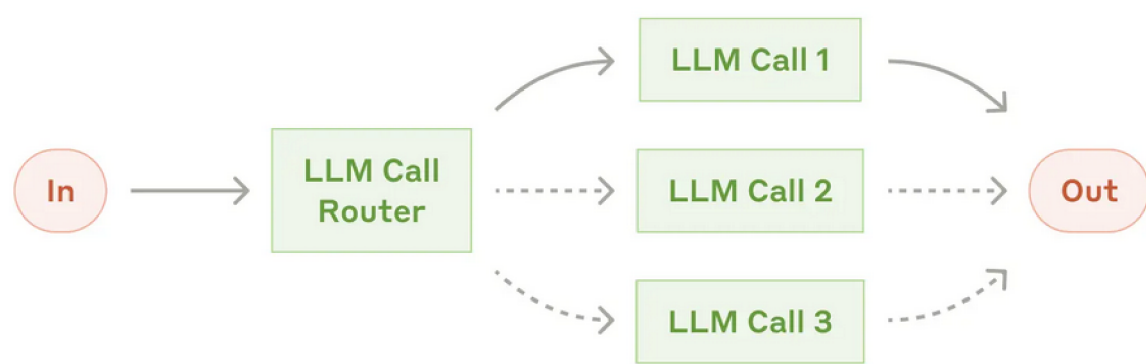
LLMs can be augmented with real world data sources, function endpoints, or caches to enable them to be agents, termed the **Augmented LLM**, which is a basic building block in this new age of agentic systems.

Types of Agentic Patterns

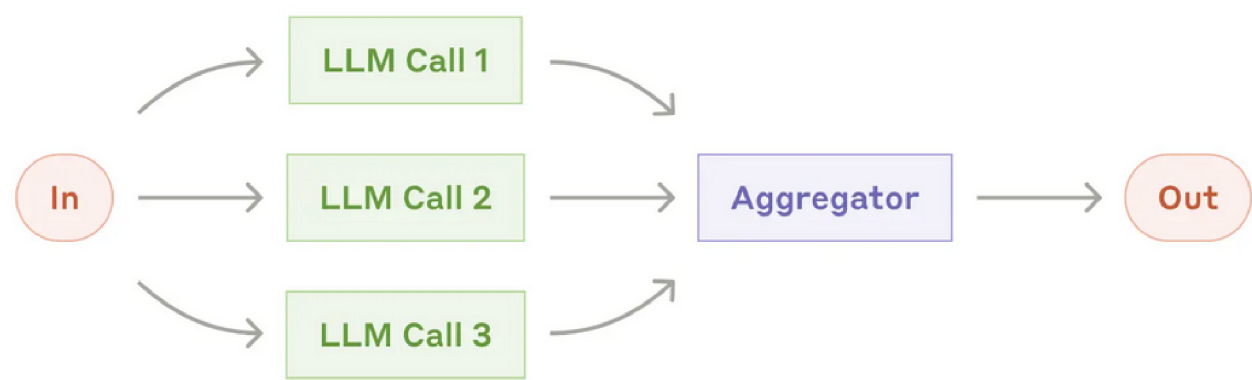
The Prompt Chain



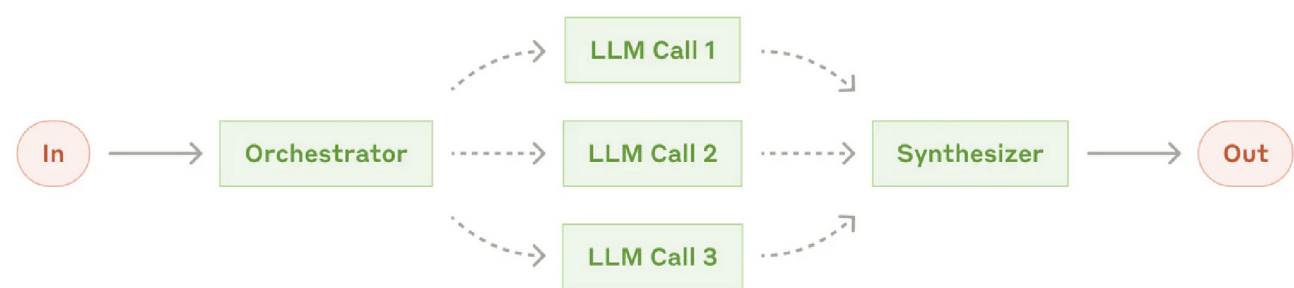
The Router



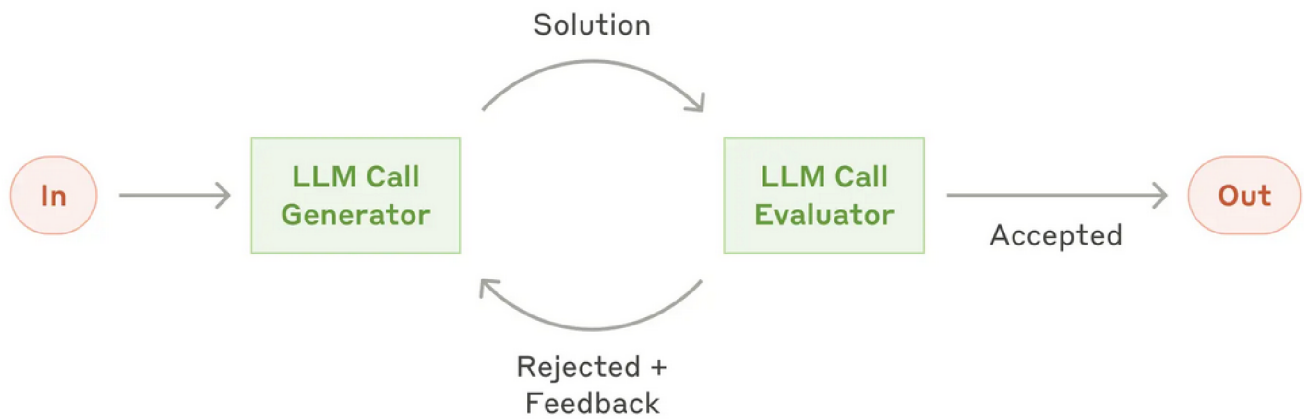
The Parallelization



The Orchestrator



The Evaluator-Optimizer



MCP

Model Context Protocol defines an implementation guideline for client-server communication, where the client is an agent, and the server provides contextual information about additional capabilities, such as tools it can use or resources it can retrieve.

A Main Idea

'Model Context' in MCP : defining a standardized way to feed context into an LLM.

Standardizing how we feed context into an LLM is akin to prompt engineering. When we let an LLM have multi-turn reasoning and allow it to evoke actuators (tools), it becomes an *agent*.

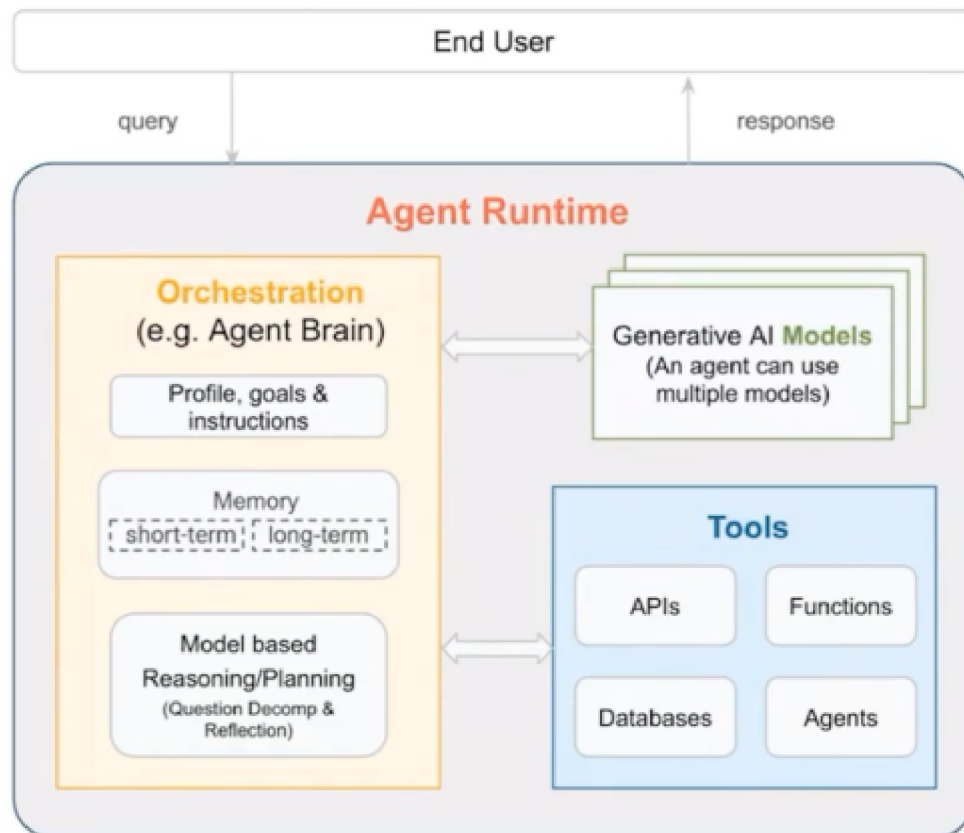
01 Implementation

we give the agent specific prompts that inform the agent about the functions that it can call, making sure to also use semantically well-chosen declaration, parameter names, doc strings, and utilizing python type hints (and even pydantic models/typed dictionaries.)

Server → REST-like API with semantic details

Client → LLM with agentic capabilities (multi-turn reasoning)

Program → Tasking an agent to perform an action.



Server

The server is responsible for providing the following to an agent:

- ① **Tools** - computational functions (file operations, API calls, database queries)
- ② **Resources** - read-only functions (file reads, API response, database reads)
- ③ **Prompts** - Structured Strings ("Utilize {query} and attempt {tool}")

Initializing a server is done with JSON to define the server name, its capabilities, and how to execute it:

```
# Registering MCP servers in VScode
{
  "servers": {
    "mcp-blog": {
      "type": "stdio",
      "command": "uv",
      "args": [
        "run",
        "python",
        "servers/mcp-blog.py"
      ]
    },
    "mcp-files": {
```

```

        "type": "stdio",
        "command": "uv",
        "args": [
            "run",
            "python",
            "servers/mcp-files.py"
        ]
    },
    "inputs": []
}

```

Scaling/Managing Available Tools

We would like to give the agent access to (realistically) every software we have built, but that would involve spinning up thousands of web servers for the agent to interact with.

Servers can be lazy initialized, as in the client sees the command call to start the server. MCP has implementation guidelines for defining lifetimes & authentication, which can be used to smartly spin up or down servers as needed; allowing frequent tools (like a web searcher) to stay always on, but something like a specific bioinformatic toolchain tool can stay offline until the workflow is required.

Tools

Tools are the meat of MCP, which allows an LLM to interact with and execute real code.

Tool Use

LLMs have their system prompt injected with context about the 'tools' available to them. The context of a tool is created through variable typing (params & return), the function signature, and the doc string.

```

@mcp.tool()
async def create_post(title: str, content: str, tags: List[str]) → str:
    """
    Create a new blog post.
    @param: title - post title string
    @param: content - body of post, formatted in markdown

```

```
@param: tags - list of strings that represent topic area
@return - success/failure message
"""
```

Resources

Resources define a 'ground truth' of documents that the agent is able to reference.

```
@mcp.resource("/guides/{post_id}")
def read_tutorials(post_id: int) → str:
    """ Reads the document with the given id """
    return db.posts.get(post_id)
```

Prompts

Prompts are reusable templates for using the agent in an interactive client. For example, if I want to ask the agent to answer my question by retrieving the relevant document by using the `read_tutorial` tool, then instead of specifying all that detail we can create a prompt that takes in an argument of the topic of the post to query:

```
@mcp.prompt(title="Summarize Information Prompt")
def summarize_information_prompt(prompt: str) → str:
    """Return a system prompt for summarizing information."""
    system_prompt = f" \
        Summarize the information from the context given by the tool
`summarize_information` \
        To give a concise summary of the key points about the topic, \
        Use the Topic: '{prompt}' as the parameter for
`summarize_information`."
    return system_prompt
```

Clients

The client for an MCP server is just a program that can instantiate LLMs. Therefore the most common clients out right now are various IDEs such as Cursor, VScode, Warp, etc, with the standard being Anthropic's Claude Desktop.

The client also has resources that it can expose, specifically Sampling & Elicitation.

Sampling

Sampling is used to generate tokens from an LLM inside of a function call. You can have a flow where the user enters a topic, and the tool generates a poem about said topic.

```
@mcp.tool()
def generate_poem(topic: str) → str:
    prompt = f"Generate a poem on {topic}"
    result = await ctx.session.create_message(
        messages=[
            SamplingMessage(
                role="user",
                content=TextContent(type="text", text=prompt)
            )
        ],
        max_tokens=100,
    )
    # result is a TextContent
    return result.content.text
```

Elicitation

Elicitation is when a tool use requires additional human input, such as prompting the user to enter a number for the number of iterations to run a loop for. In my opinion, I think you can find a way to rewrite the function so that whatever human input the LLM would need at runtime, it can instead be given at the beginning as a parameter.

Implementation Notes

Prompt Engineering in Disguise

Writing APIs has become so boiler plate, that even Anthropic themselves says to just write a specification and let the LLM/copilot create all the code for the servers.

The actual coding is coming up with ways to reduce the variability in the AI; An anecdote -> I tried to have a toolchain of `Document summarizer` → `markdown_to_html formatter` → `blog poster`, however the *context from the tool response* can override the system prompt, and cause mass confusion in the AI and downstream agents. The `markdown_to_html` tool would take in a markdown summary and return html, but the returned raw html would cause the agent to not know what to do with it, and the resulting `blog poster` would also crumble.

⚡ Token Use

The other caveat is the amount of tokens that is consumed just to save a user from filling out a form. For simple tools, they utilize a lot more token than an average interactive chat, upwards to 25,000 tokens just to read a document and call an API endpoint. The tool should be worthwhile in the lens of power consumption.

⚠️ Antithesis of Programming

The amount of variability in all aspects of development:

- ① The model you choose
- ② The endpoint implementations
- ③ The LLM's temperature

All cause unexpected behaviors, resulting in a nightmare programming experience → is the bug your experiencing a hallucination or a real runtime error? Is the prompt you are feeding it not descriptive enough, or is this model just not good at reasoning about tool use?

✓ 90% There

The biggest reason I think MCP is gaining so much traction and is sparking so much attention, is that something that only a VC startup could achieve 5 years ago, can be realistically achieved to 90% of that goal with a simple MCP server → Devin was just an AI agent that tool access to a browser, terminal, and IDE, and was a billion dollar project with top talent, but now anyone can achieve 90% of the expected behavior of a 'coding assistant' with just simple MCP tools.

Summary

Model Context Protocol provides a standard way to define an API so that an LLM can interact with it as efficiently as possible; you define 'tools' that the model can use, which provides context to the model about the tool use.

The purpose ; MCP enables a full orchestration of a typical human's flow → if a user needs to first check some document, then select some entries according to a constraint, and finally

format their findings, this entire flow can be captured with a single query and the LLM can execute everything it needs to with the given tools like `list_recent_sales()`, `apply_statistic()`, `format_summary()`.

RAG

Retrieval Augmented Generation is a design pattern to let an LLM connect to a data source.

Query → Retriever → Top-k Chunks → Prompt template → Response

We have a set of document, say pdfs or markdown files, and we want to create a new database for our files of interest. To create a database that an LLM can interact with, we utilize **embeddings** and store embeddings in a vector store.

A Embeddings

Embedding: Storing a word as a **SEQUENCE** of integers, similar to a **key:value** pair where **word:embedding** and the embedding is a *unique identifier* for the given word.

Embeddings are how the LLM 'learns' what words *mean*. This is what differentiates an LLM that can 'understand' code versus novels → it will learn the embedding (identifier) for the whitespace token, and have a very different meaning for each model.

Therefore to setup a RAG, we want to create new embeddings for our files → our documents have a specific meaning, and we don't want the AI to confuse our meanings with what it has already learned.

Document Loader → Text splitter → Embeddings → Vector Store

Creating Embeddings

Document Loader + Text Splitter

When we create our tokens to be embedded, we want the tokens to have a very deep **semantic meaning** to help the model learn as accurately as possible. Therefore, if we are trying to RAG our notes, we want to split up our tokens into paragraphs where each paragraph captures a single idea.

This can be done with *another AI* to parse through our documents and create these semantic chunks.

Learning Embeddings

Embeddings + Vector Store

We then want to train a model that only learns these new embeddings → we don't want to have to retrain an entire model to have it relearn english, we only need it to learn the new semantic meanings specific to our documents. Once we train for a while and learn good embeddings, we need to store these in a database.

Vector databases

Remember that arrays $\leftrightarrow$ vectors, so databases designed to only store large arrays are coined **Vector Databases**. Additionally, these perform a *cosine similarity* (or a dot product) between two vectors; the query and the stored entries. This allows us to take those semantic chunks which may contain words like "vector operations", "matrices", "magnitude" which would have a (hopefully) high similarity to the query 'vector'.

Using Embeddings

Now, we need to allow an existing chatbot to be able to either use its default behavior OR use our new embeddings from our database; if we are still asking it normal questions ("How are you"), we want it to still respond how it usually would, but if we are asking it to retrieve something specific from our source ("What did last weeks email say about this weeks meeting?"), instead of using its normal embeddings, *it switches to use our embeddings stored in the vector database*.

Implementation

LangChain is the standard library for creating RAG flows. It has each of the steps preloaded into functions that can be hotswapped to use with any model or database.

MCP also allows for an easy implementation as it can still behave like the original model, with the addition of quering a vector database for additional context to use in its answer.

Website

This website is going to *stupid simple* : I just want to be able to post my content to the internet, the actual design doesn't entirely matter.

- Django (comfortable) server (could even just use flask because how simple it can be → there isn't even a data model that is necessary, just loading templates)
- HTMX/CSS Frontend (make reactive for inserting new posts)

I can create a pipeline "find my note on linear regression and add it to my blog and tweet about it" and the AI can

- ① Utilize a RAG to find my linear regression document
 - ② Change the formatting from markdown to html
 - ③ Save the html to a new file
 - ④ Call a function to add a new post to the website
 - ⑤ Write a 2 line summary about the blog post
 - ⑥ Post the summary to twitter
-